

WEEK 8 | SCALABLE TRAINING + DEPLOYMENT

Checkpointing & Fault Tolerance

Saving state without stalling your GPUs; resuming without losing days of work.

Satyadhyan Chickerur, Ph.D., MBA(Edu.Mgmt), SMIEEE
Director, Centre for Artificial Intelligence Research,
Prof., School of Computer Science & Engineering,
KLE Technological University,
BVB Campus, Vidyanagar,
Hubballi - 580031, Karnataka, India

The Big Picture — Why This Session Exists

Imagine spending a week and thousands of dollars training a model, only for a single failed GPU on day 6 to wipe out all your progress.

THINK OF IT LIKE THIS: Checkpointing is like saving your progress in a long video game before a difficult boss fight. Die (a GPU fails) after saving recently, and you reload, losing minutes. Never save, and you're back at the start — hours of progress, gone.

1

Failures at scale are guaranteed

Enough GPUs, running long enough — something WILL fail. Statistical certainty, not bad luck.

2

A checkpoint must capture everything

Miss one piece of state and your 'resume' quietly trains worse than it should.

3

Saving costs time — do it smartly

Naive saving stalls expensive GPUs; techniques exist to make it nearly free.

Key Terms Before We Start

A quick shared vocabulary — skip ahead if these are all second nature to you.

Checkpoint — A saved snapshot of everything needed to resume training exactly where it left off.

Node / worker / rank — A node is one machine; a worker (rank) is one training process, usually one per GPU.

Fault tolerance — A system's ability to keep working (or recover quickly) when part of it fails.

Sharding — Splitting one big thing (like a checkpoint) into pieces, each handled by a different worker.

Elastic training — Training that can keep running even as workers join or leave, without a full restart.

Rendezvous — The process by which distributed workers find and connect to each other at startup.

Failure Rates at Scale

22%

CHANCE OF ANY

node failing today (256 nodes)

- GPU falls off PCIe — driver loses the device, node needs a reboot
- NCCL hang — one rank dies silently, others block forever
- OOM — memory fragmentation or a mis-estimated budget
- Spot preemption — cloud yanks the node back with little warning
- Cluster maintenance — a Kubernetes upgrade drains your pod mid-run

THINK OF IT LIKE THIS: Rolling a thousand dice every day, hoping none show a 1. With enough dice (nodes), something rolls a 1 almost every day. Plan for it like rain on a long road trip.

Why 'Rare' Failures Become 'Guaranteed' at Scale

Any single component might be extremely reliable. The problem is thousands of them running at once, for a long time.

```
p_fail_per_node_per_day = 0.001      # 0.1% – each node is quite reliable
num_nodes = 256
p_no_fail_today = (1 - p_fail_per_node_per_day) ** num_nodes    # ~0.776
p_at_least_one = 1 - p_no_fail_today                               # ~22%

# Over 14 days, the odds compound further:
p_no_fail_2wk = p_no_fail_today ** 14      # ~2.6%
p_at_least_one_2wk = 1 - p_no_fail_2wk     # ~97.4%
```

Same math as 'why my 200-person office always has someone out sick.' Individually low odds, multiplied across many nodes and days, become near-certain.

What a Correct Checkpoint Contains

Model weights

The learned parameters — `model.module.state_dict()` under DDP.

Optimizer state

Adam's running averages. Forgotten often — convergence quietly regresses.

LR scheduler state

Current step, warmup progress, cosine phase.

Step + epoch

So you resume at step+1, not step 0.

RNG state

Needed for bit-identical reproducibility.

Sampler + AMP scaler

Which examples were seen; the fp16 loss scale.

FAILURE MODE: most 'resume' bugs come from forgetting the last 3-4 items. Loss looks fine; convergence is quietly off.

Walking Through a Save/Resume Cycle

Let's trace exactly what happens, in order, when a job fails and comes back.

- 1 Training reaches step 700. Every N steps, `save()` writes `weights+optimizer+scheduler+step+RNG+sampler`
- 2 The checkpoint is written to storage, tagged with its step number
- 3 At step 743, one worker's GPU fails – the process crashes
- 4 The launcher (e.g. `torchrun`) notices the missing worker and restarts the job
- 5 On restart, the script finds the LATEST valid checkpoint – step 700
- 6 It loads ALL saved state and sets the global step counter to 701, not 0
- 7 Training resumes as if the crash never happened – you lose only 43 steps

Sharded vs Full Checkpoints

Full checkpoint

Single file, complete state.
Simple/portable but every rank gathers to rank 0 — minutes of GPU idle per save at scale.

Sharded checkpoint

Each rank writes its own piece, in parallel — 10-100x faster. Needs coordinated-load logic to reshard.

THINK OF IT LIKE THIS: Full = one person photocopying a 1,000-page book alone. Sharded = 10 people each photocopy their own chapter simultaneously — faster, but now you need a plan to reassemble the book.

```
from torch.distributed.checkpoint import save, load, FileSystemReader,
FileSystemWriter
state = {'model': model.state_dict(), 'optim': optim.state_dict(), 'step':
global_step}
save(state, storage_writer=FileSystemWriter('/ckpt/step-1000'))    # SAVE, parallel
load(state, storage_reader=FileSystemReader('/ckpt/step-1000'))    # LOAD, auto-
reshards
```

Async Checkpointing — Don't Stall the GPUs

A synchronous save can take 30s-5min at scale. During that time, every expensive GPU sits idle.

THINK OF IT LIKE THIS: Method 1: drive the package yourself, blocked until you're back. Method 2: hand it to a courier and keep working. Async checkpointing hands the 'save' to a background process so training keeps going.

```
from torch.distributed.checkpoint import async_save
future = async_save(state, storage_writer=FileSystemWriter('/ckpt/step-1000'))
# training continues immediately, using the GPUs, while the save happens in
background
future.result()    # only wait for THIS before starting the NEXT save
```

PyTorch 2.2+. In practice: ~60 seconds down to ~2 seconds — nearly free.

Elastic Training — Losing (and Gaining) Workers

'Elastic' means the job survives nodes joining or leaving without a full restart from scratch.

THINK OF IT LIKE THIS: A relay race team that keeps racing even if a runner drops out — the baton (training state) is picked up by whoever's left.

```
torchrun --nnodes=1:8 --nproc_per_node=8 \  
  --rdzv_backend=c10d --rdzv_endpoint=head:29500 --rdzv_id=my-job \  
  --max_restarts=3 train.py
```

THE HARD PART: your code must be restart-safe — idempotent data loading, checkpoint-on-interrupt, correct step accounting after a topology change. A discipline, not a flag.

Checkpoint Frequency — the Cost/Risk Trade-off

More frequent checkpoints lose less work on a crash but cost more I/O. Async checkpointing shifts the balance toward frequent saves.

Every	Max work lost	Sync overhead	Async overhead
100 steps	~100 steps	~60%	~2%
500 steps	~500 steps	~12%	~0.4%
1000 steps	~1000 steps	~6%	~0.2%

PRACTICAL DEFAULT: every 500-1000 steps with async checkpointing on. Tighten to 100 only for known-flaky clusters.

Common Mistakes & How to Spot Them

Mistake	How it shows up	Fix
Not saving optim/sched/RNG state	Resumed training converges worse than uninterrupted	One save()/load() covering every piece of state, always
Full checkpoints at huge scale	Every save stalls GPUs for minutes	Switch to sharded checkpoints (DCP)
Checkpointing too rarely on flaky clusters	A single crash wipes out hours	Increase frequency; enable async checkpointing
Assuming 'elastic' needs no code changes	Job crashes hard on first node loss	Build restart-safe, idempotent loading logic

Quick Recap — Terms and Concepts

Checkpoint — A saved snapshot of ALL training state

Sharded checkpoint — Each worker saves its own piece, in parallel

Async checkpointing — Copy to fast memory first, write to disk in the background

Elastic training — Job survives workers joining/leaving, given restart-safe code

Checkpoint frequency — A trade-off between I/O overhead and work lost on a crash

Session 2 Summary

- At 256 nodes with a 0.1%/day failure rate, ~22% chance of ANY node failing today — plan for it
- A correct checkpoint saves ALL state: weights, optimizer, scheduler, step/epoch, RNG, sampler, AMP scaler
- Sharded checkpoints write in parallel and reshard automatically on resume
- Async checkpointing hides the disk write behind a fast pinned-memory copy — ~60s down to ~2s
- Elastic training survives node loss but requires restart-safe, idempotent code

NEXT: Model Packaging & Versioning — safetensors, ONNX, MLflow registry.